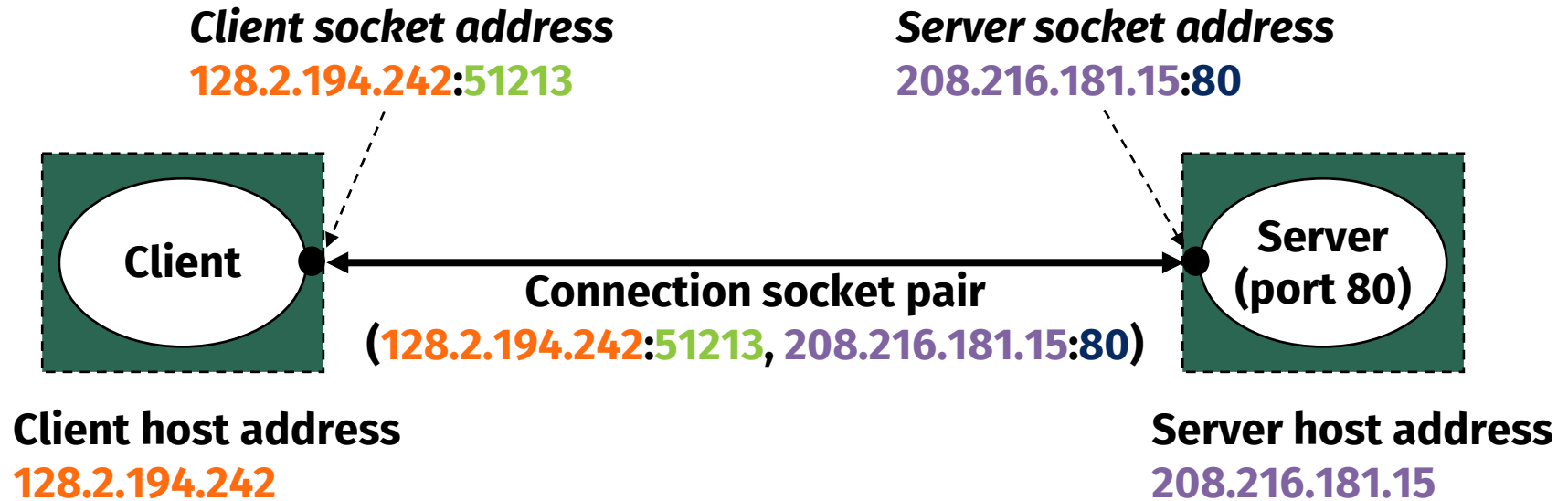


Sockets

Internet Connections (1)

- **Clients and servers communicate by sending streams of bytes over connections:**
 - Point-to-point, full-duplex, and reliable
- **A socket is an endpoint of a connection**
 - A socket address is an <IP address : port number> pair
- **A port is a 16-bit integer that identifies a process**
 - **Ephemeral port:** assigned automatically to the client when client makes a connection request
 - **Well-known port:** associated with some service provided by a server (e.g., port 80 is associated with web servers)
- **A connection is uniquely identified by the socket addresses of its endpoints (socket pair)**
 - <client IP:client port no., server IP:server port no.>

Internet Connections (2)

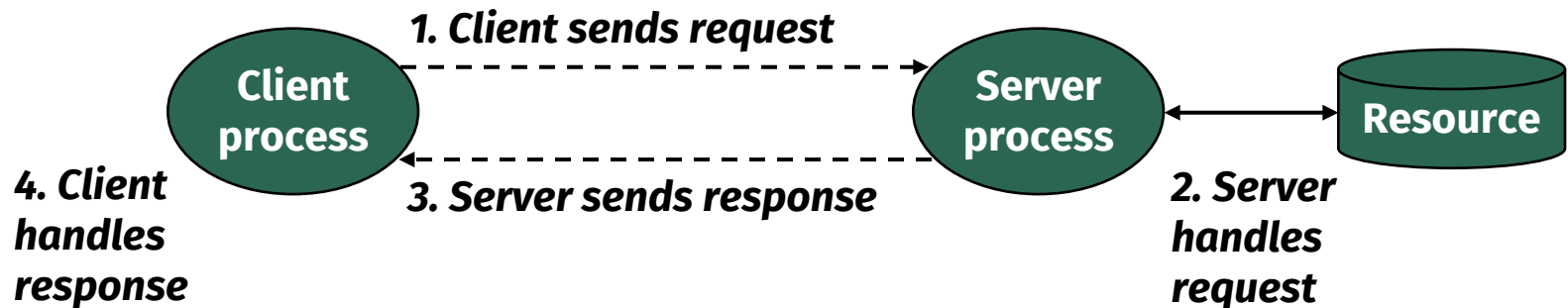


Note: 51213 is an ephemeral port allocated by the kernel

Note: 80 is a well-known port associated with Web servers

Client-Server Model

- **Most network application is based on the *client-server model***
 - A **server** process and one or more **client** processes
 - » Clients and servers are processes running on hosts (can be the same or different hosts)
 - Server manages some **resource**
 - Server provides **service** by manipulating resource for client requests



Clients

- **Examples of client programs**

- Web browsers, ftp, telnet, ssh, etc.

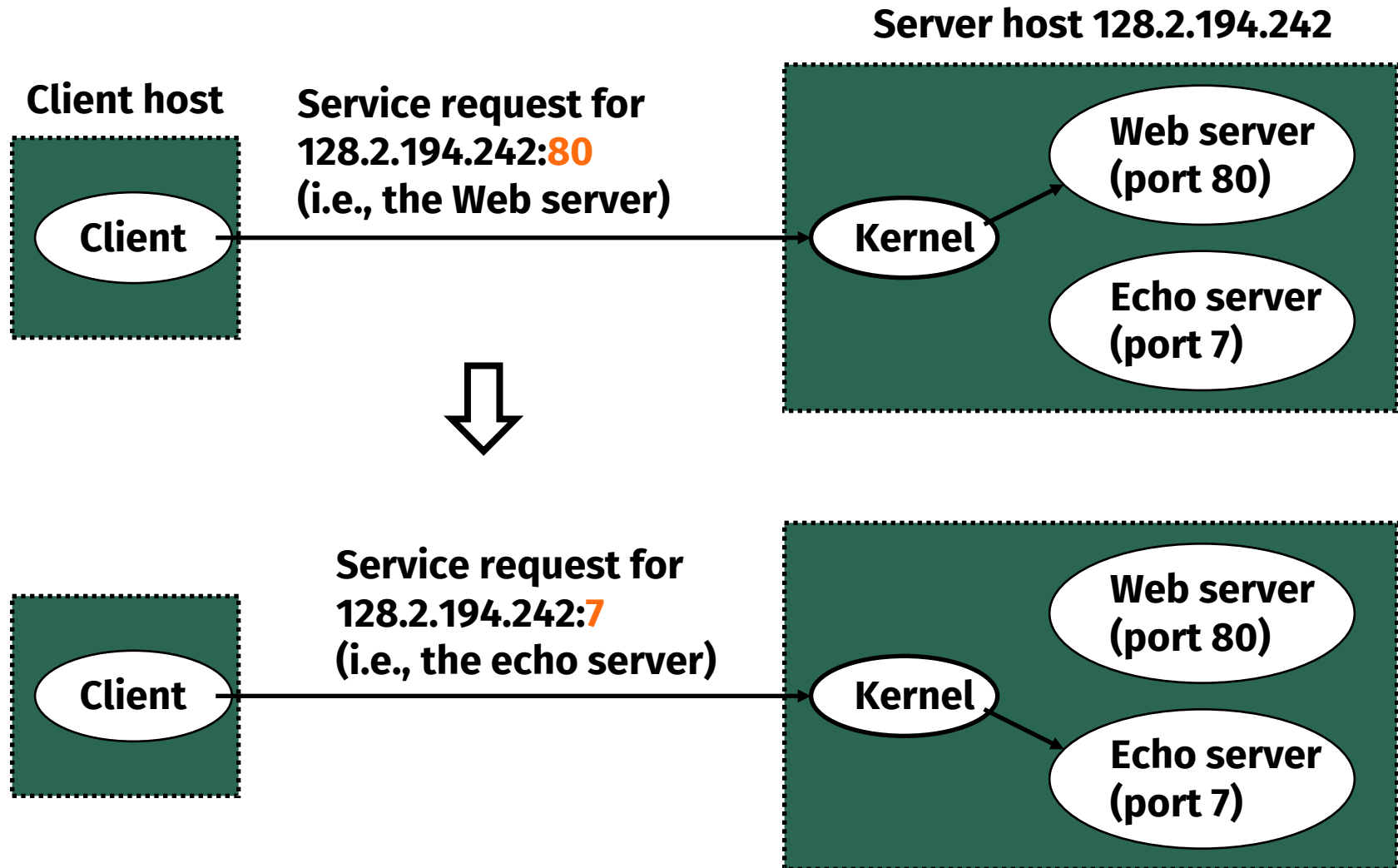
- **How does a client find the server?**

- The **IP address** in the server socket address identifies the host
- The **(well-known) port** in the server socket identifies the service; thus, implicitly identifies the server process that performs that service
- Examples of well-known ports (cf. */etc/services*)

- » Port 21: ftp
- » Port 22: ssh
- » Port 23: telnet
- » Port 25: mail (smtp)
- » Port 80: web (http)

ftp	21/tcp	
fsp	21/udp	fspd
ssh	22/tcp	# SSH Remote Login Protocol
telnet	23/tcp	
smtp	25/tcp	mail
time	37/tcp	timserver
time	37/udp	timserver
whois	43/tcp	nicname
tacacs	49/tcp	# Login Host Protocol (TACACS)
tacacs	49/udp	
domain	53/tcp	# Domain Name Server
domain	53/udp	
bootps	67/udp	
bootpc	68/udp	
tftp	69/udp	
gopher	70/tcp	# Internet Gopher
finger	79/tcp	
http	80/tcp	www # WorldWideWeb HTTP

Using Ports



Servers

- **Servers are long-running processes (daemons)**
 - Created at boot-time (typically) by the init process (process 1)
 - Run continuously until the machine is turned off
- **Each server waits for requests to arrive on a well-known port associated with a particular service**
 - e.g. port 21 (FTP), port 23 (Telnet), port 80 (HTTP)
- **A machine that runs a server process is also often referred to as a “server”**

Socket (1)

▪ Sockets interface

- Was introduced in BSD4.1 UNIX, 1981
- Is based on **client/server paradigm**
- Provides a user-level interface to the network
- Can be explicitly created, used, released by applications
- Consist of two types of transport service
 - » **UDP**: Unreliable, fast datagram
 - » **TCP**: Reliable, connection-oriented byte stream
- Is the underlying basis for all Internet applications

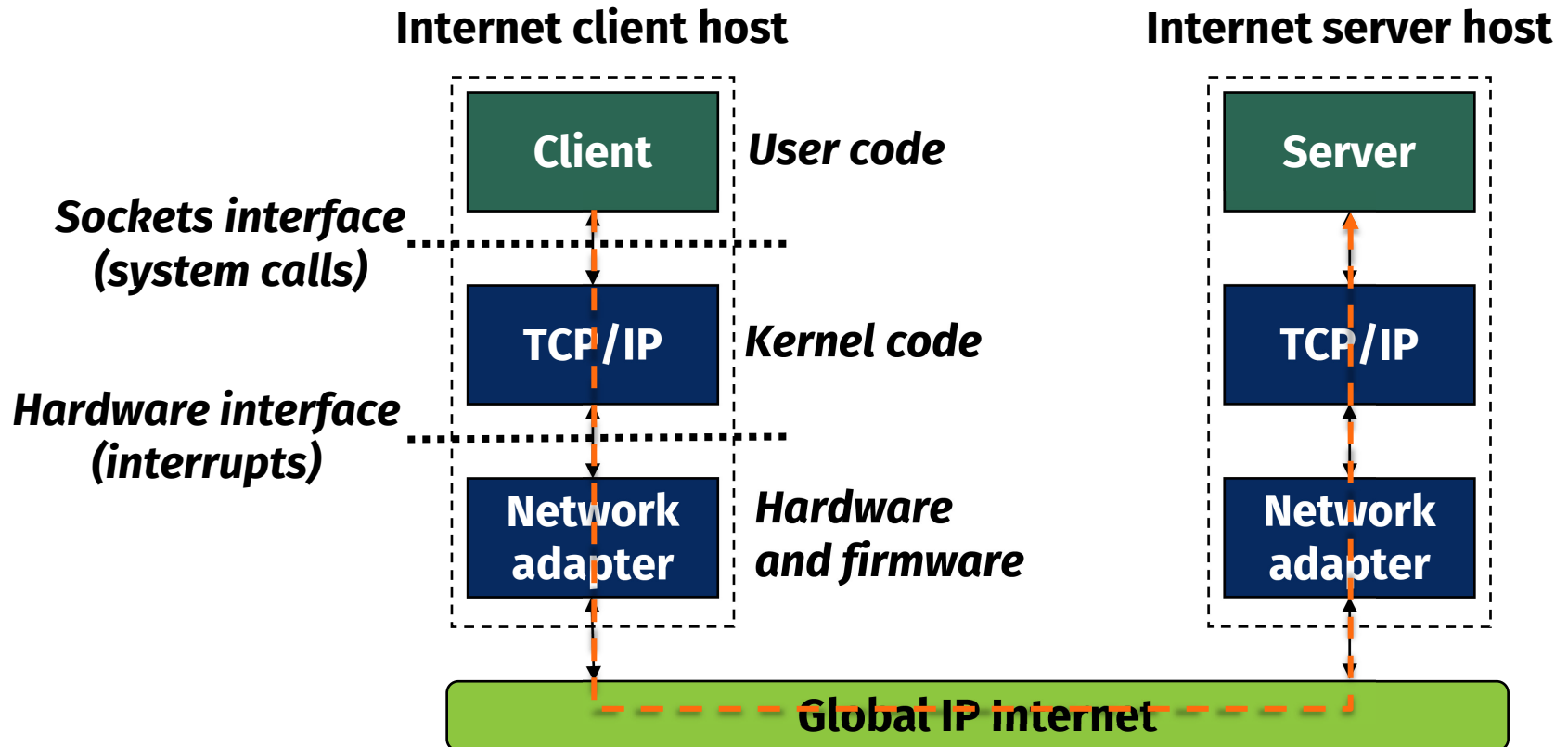
Socket (2)

■ What is a socket?

- A remote-local, application-created/owned, OS-controlled interface to network (a “door”)
 - » To the kernel, a socket is an endpoint of communication
 - » To an application, a socket is a file descriptor
 - » Applications read/write from/to the network **using the file descriptor**
 - » Note: All Unix I/O devices, including networks, are modeled **as files**
- Clients and servers communicate with each by reading from and writing to socket descriptors
 - » The main distinction between regular file I/O and socket I/O is how the application “opens” the socket descriptors

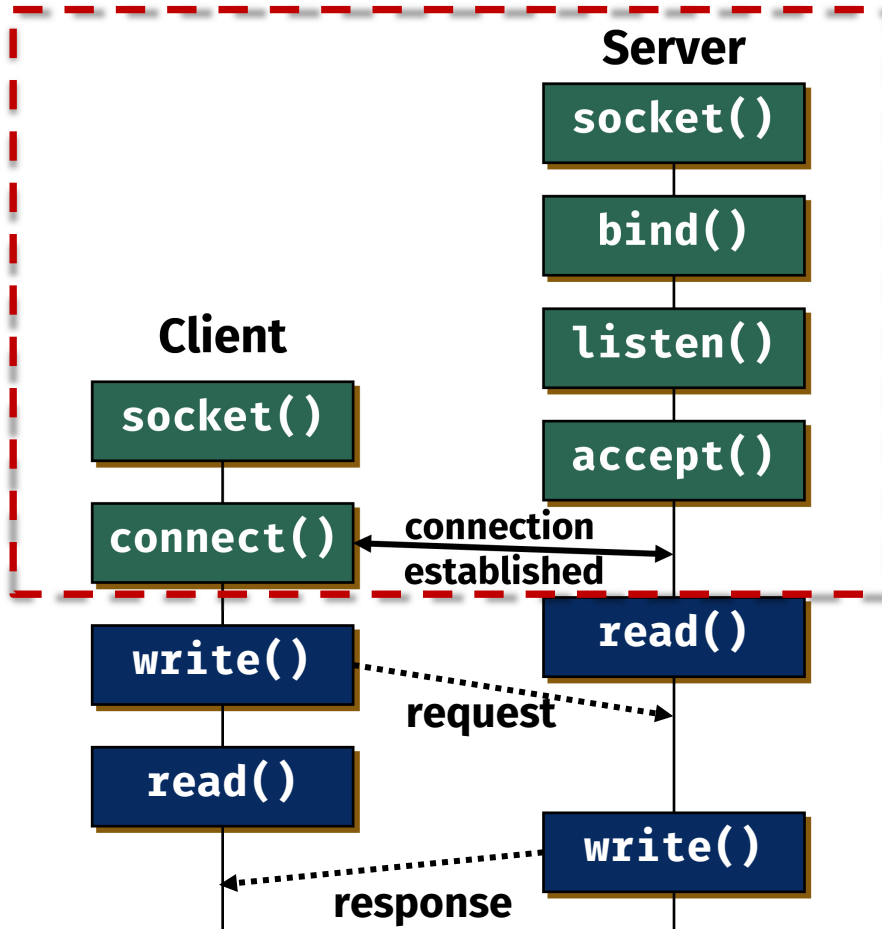
Socket (3)

- Hardware/Software organization of an Internet application

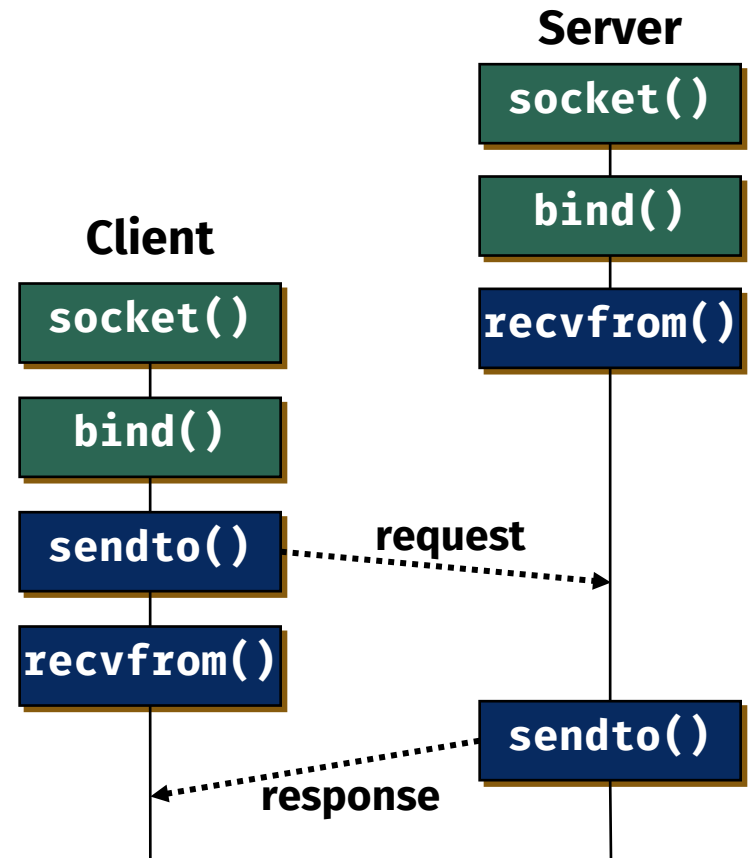


TCP vs. UDP

Connection-oriented service



Connectionless service



Socket Address Structure

■ Generic socket address

- For address arguments to **connect()**, **bind()**, and **accept()**

```
#include <sys/socket.h>
struct sockaddr {
    unsigned short sa_family; /* address family, (e.g. AF_INET) */
    char sa_data[14];        /* address data */
};
```

■ Internet-specific socket address

- Must cast (**sockaddr_in***) to (**sockaddr***)
for **connect()**, **bind()**, and **accept()**

```
#include <netinet/in.h>
struct sockaddr_in {
    sa_family_t sin_family; /* address family (i.e., AF_INET) */
    in_port_t sin_port;     /* port num in network byte order */
    struct in_addr sin_addr; /* IP address in network byte order */
    unsigned char sin_zero[8]; /* pad to sizeof(struct sockaddr) */
};
```

socket()

- `int socket(int domain, int type, int protocol)`
 - **socket()** creates a socket descriptor
 - **domain** specifies the communication domain
 - » **AF_UNIX**: Local Unix domain protocol (associated with UDS)
 - » **AF_INET**: IPv4 Internet protocol
 - » **AF_INET6**: IPv6 Internet protocol
 - **type** specifies the communication semantics
 - » **SOCK_STREAM**: provides sequenced, reliable, two-way, connection-based **byte streams**
 - » **SOCK_DGRAM**: supports **datagrams** (connectionless, unreliable messages of a fixed maximum length)
 - » **SOCK_RAW**: provides raw network protocol access
 - **protocol** specifies a particular protocol to be used with the socket (`IPPROTO_IP`, `IPPROTO_TCP`, `IPPROTO_UDP`)

connect()

- `int connect(int sockfd,
 const struct sockaddr* addr,
 socklen_t addrlen)`
 - Used by a **connection-based client** (i.e., TCP and UDS, but not UDP) to establish a connection with a TCP server
 - **addr** contains **<IP address, port number>** of the server
 - The client does not have to call **bind()** before calling **connect()**
 - » The kernel will choose both an ephemeral port and the source IP address if necessary
 - Client process **suspends (blocks)** until the connection is created.

bind()

- `int bind(int sockfd, const struct sockaddr* addr, socklen_t addrlen)`
 - ***bind()*** gives the socket ***sockfd*** the local address ***addr***
 - ***addr*** is ***addrlen*** bytes long
 - Servers bind their well-known port when they start
 - If a TCP server binds a specific IP address to its socket, this **restricts** the socket to receive incoming client connections **destined only to that IP address**
 - Normally, a TCP client let the kernel choose an ephemeral port and a client IP address

listen()

- `int listen(int sockfd, int backlog)`
 - **listen()** converts an unconnected socket into a **passive socket**, indicating that the **kernel should accept incoming connection requests**
 - » When a socket is created, it is assumed to be an active socket, that is, a client socket that will issue a **connect()**
 - **backlog** specifies **the maximum number of connections** that the kernel should queue for this socket
 - Historically, a backlog of 5 was used, as that was the maximum value supported by 4.2BSD
 - » Busy HTTP servers must specify a much larger backlog, and newer kernels must support larger values

accept() (1)

- `int accept(int sockfd,
 struct sockaddr* addr,
 socklen_t* addrlen)`
 - **accept()** **blocks waiting** for a connection request
 - **accept()** returns a **connected descriptor** with the same properties as the **listening descriptor**
 - » The kernel creates one connected socket for each client connection that is accepted
 - » Returns when **the connection** between client and server is **created and ready for I/O transfers**
 - » All I/O with the client will be done via the connected socket
 - The **addr** and **addrlen** arguments are used to return the address of the **connected peer process** (the client)

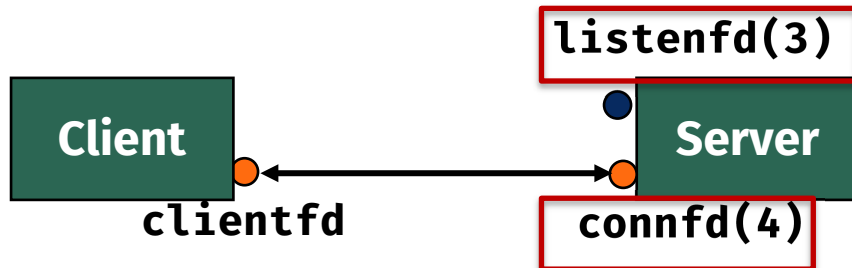
accept() (2)



1. Server blocks in *accept*, waiting for connection request on listening descriptor *listenfd*.



2. Client makes connection request by calling and blocking in *connect*.



3. Server returns *connfd* from *accept*. Client returns from *connect*. Connection is now established between *clientfd* and *connfd*.

accept() (3)

▪ Listening descriptor

- End point for client connection requests
- Created once and **exists for lifetime of the server**

▪ Connected descriptor

- End point of the **connection between client and server**
- **A new descriptor** is created **each** time the server accepts a connection request from a **client**
- Exists only as long as it takes to service client

▪ Why the distinction?

- Allows for **concurrent servers** that can communicate over many client connections simultaneously

Echo Client (0)

memset(3)

Library Functions Manual

NAME

memset - fill memory with a constant byte

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

#include <string.h>

void *memset(void s[], int c, size_t n);

DESCRIPTION

The `memset()` function fills the first n bytes of the memory area pointed to by s with the constant byte c.

RETURN VALUE

The `memset()` function returns a pointer to the memory area s.

memcpy(3)

Library Functions Manual

memcpy(3)

NAME

memcpy - copy memory area

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

#include <string.h>

void *memcpy(void dest[restrict .n], const void src[restrict .n],
size_t n);

DESCRIPTION

The `memcpy()` function copies n bytes from memory area src to memory area dest. The memory areas must not overlap. Use `memmove(3)` if the memory areas do overlap.

RETURN VALUE

The `memcpy()` function returns a pointer to dest.

Echo Client (1)

```
#include <arpa/inet.h>
#include <errno.h>
#include <netdb.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>
#define MAXLINE 80
```

```
int main(int argc, char* argv[]) {
    long n;
    int conn_fd;
    struct hostent* host_entry;
    struct sockaddr_in sock_addr;
    char buf[MAXLINE];

    if (argc != 3) {
        printf("usage: %s <host> <port>\n", argv[0]);
        exit(1);
    }

    char* host = argv[1];
    long port = strtol(argv[2], NULL, 10);
    if (errno == ERANGE || port < 0 || port > 65535) {
        printf("invalid port number %s\n", argv[2]);
        exit(1);
    }

    if ((conn_fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP)) < 0) {
        printf("socket() failed.\n");
        exit(2);
    }
}
```

Echo Client (2)

```
if ((host_entry = gethostbyname(host)) == NULL) {
    printf("invalid hostname %s\n", host);
    exit(3);
}

memset(&sock_addr, 0, sizeof(sock_addr));
sock_addr.sin_family = AF_INET;
memcpy(&sock_addr.sin_addr.s_addr,
       host_entry->h_addr_list[0], host_entry->h_length);
sock_addr.sin_port = htons(port);

if (connect(conn_fd, &sock_addr, sizeof(saddr)) < 0) {
    printf("connect() failed.\n");
    exit(4);
}
while ((n = read(STDIN_FILENO, buf, MAXLINE)) > 0) {
    write(conn_fd, buf, n);
    n = read(conn_fd, buf, MAXLINE);
    write(STDOUT_FILENO, buf, n);
}
close(conn_fd);
}
```

Echo Server (1)

```
int main(int argc, char *argv[]) {
    long n;
    int listen_fd, conn_fd;
    socklen_t conn_addr_len;
    struct sockaddr_in sock_addr, conn_addr;
    char buf[MAXLINE];
```

```
    if (argc != 2) {
        printf("usage: %s <port>\n", argv[0]);
        exit(1);
    }
```

```
    long port = strtol(argv[1], NULL, 10);
    if (errno == ERANGE || port < 0 || port > 65535) {
        printf("invalid port number %s\n", argv[1]);
        exit(1);
    }
```

```
    if ((listen_fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP)) < 0) {
        printf("socket() failed.\n");
        exit(1);
    }
```

```
#include <arpa/inet.h>
#include <errno.h>
#include <netdb.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>
#define MAXLINE 80
```

Echo Server (2)

```
memset(&sock_addr, 0, sizeof(sock_addr));
sock_addr.sin_family = AF_INET;
sock_addr.sin_addr.s_addr = htonl(INADDR_ANY);
sock_addr.sin_port = htons(port);
if (bind(listen_fd,
        (struct sockaddr*)&sock_addr,
        sizeof(sock_addr)) < 0) {
    printf("bind() failed.\n");
    exit(2);
}
if (listen(listen_fd, 5) < 0) {
    printf("listen() failed.\n");
    exit(3);
}
```


Echo Server (3)

```
while (1) {
    conn_addr_len = sizeof(conn_addr);
    if ((conn_fd = accept(listen_fd,
                          (struct sockaddr*)&conn_addr,
                          &conn_addr_len)) < 0) {
        printf("accept() failed.\n");
        continue;
    }
    // echo
    while ((n = read(conn_fd, buf, MAXLINE)) > 0) {
        printf("got %ld bytes from client.\n", n);
        write(conn_fd, buf, n);
    }

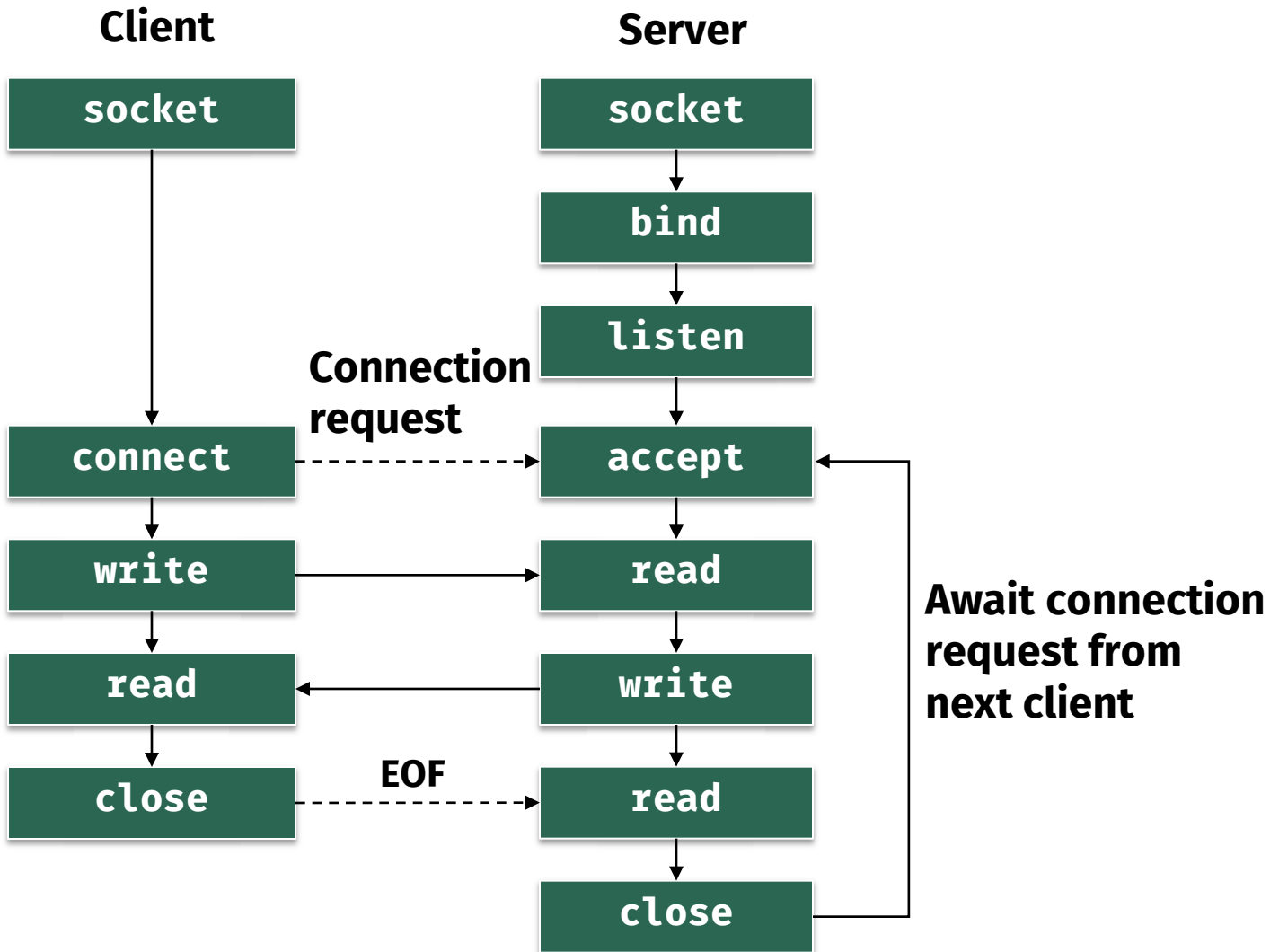
    printf("connection terminated.\n");
    close(conn_fd);
}
```

Echo Server (4)

```
🖥️ ./server 1398  
got 4 bytes from client.  
got 17 bytes from client.  
got 5 bytes from client.
```

```
🖥️ ./client localhost 1398  
Hi!  
Hi!  
안녕하세요~  
안녕하세요~  
😊  
😊
```

Echo Server (5)



Resources

- **Beej's Guide to IPC**

- <https://beej.us/guide/bgipc/html/split/>

- **Beej's Guide to Network Programming:**

- <https://beej.us/guide/bgnet/html/split/>
- Korean
 - » https://wiki.kldp.org/Translations/html/Socket_Programming-KLDP/Socket_Programming-KLDP.html
 - » https://garrettjavalia.github.io/assets/beejs_guide_to_network_programming/html/split-wide/index.html

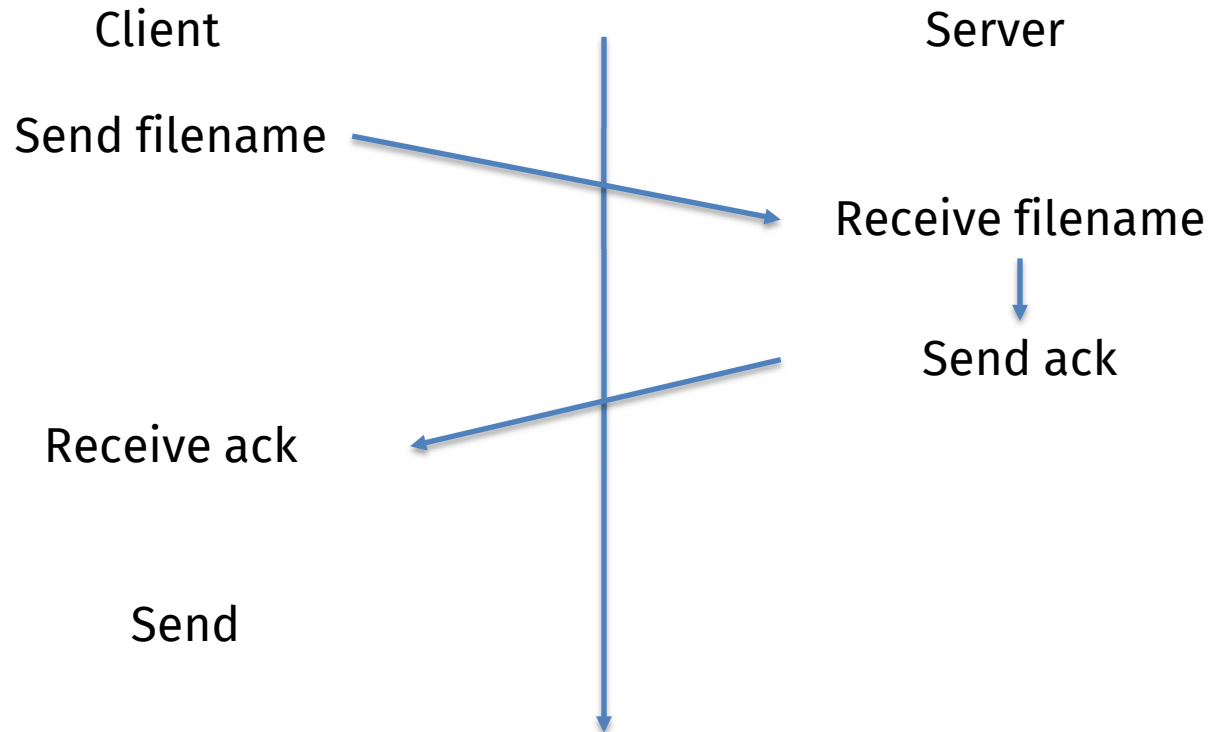
Lab Exercise

■ Make a *file transfer service*

- Use two shells: one running the server, the other running the client
- This client should be under an infinite loop with **conditional exit** (“quit”)
- When the name of the file is entered, the client reads the file and sends it to the server
- The server saves the file name received from the client by appending “_copy”
- The file is a text file and does not have an extension
 - » ex) a.txt(X), b.exe(X), c.c(X) , Aladdin(O), a(O), b(O)
- The size of file is smaller than 200,000 bytes
- The length of file name is shorter than 50 bytes.

Lab Exercise

- **Hint: Possible flow**



Lab Exercise

■ e.g.

```
spl on ● SPL in 2023f/week11/p11 via C v13.2.0-gcc
□ ls
Aladdin      client*  Makefile  server.c  text_copy
Aladdin_copy client.c  server*   text

spl on ● SPL in 2023f/week11/p11 via C v13.2.0-gcc
□ cat text
This is a text file
to test this week's exercise!↵

spl on ● SPL in 2023f/week11/p11 via C v13.2.0-gcc
□ ./client localhost 1398
File Name: text
Sending text...
Sent 49 bytes.
File Name: Aladdin
Sending Aladdin...
Sent 109511 bytes.
File Name: quit

spl on ● SPL in 2023f/week11/p11 via C v13.2.0-gcc took 8s
□
```

```
spl on ● SPL in 2023f/week11/p11 via C v13.2.0-gcc
□ ./server 1398
File Name: text
Received 49 bytes.
File Name: Aladdin
Received 109511 bytes.
^C↵

spl on ● SPL in 2023f/week11/p11 via C v13.2.0-gcc took 19s
!! stat text -c "%n: %s bytes"
text: 49 bytes

spl on ● SPL in 2023f/week11/p11 via C v13.2.0-gcc
□ stat Aladdin -c "%n: %s bytes"
Aladdin: 109511 bytes

spl on ● SPL in 2023f/week11/p11 via C v13.2.0-gcc
□
```

Exercise Submission

- **Submit your source code and Makefile via iCampus**

- Bundle *source code* and *Makefile* with tar command

- » ``cd ..`` if needed

- `$tar caf [student_id]_w10.tar.zst week10`

- We'll grade your submission with **make**

- » Should generate `./server` and `./client` executables

- » If compilation fails, your points for this exercise will be zero